

Hands-On with the SPECK Dev Kit

Mazdak Fatahi

Hardware Families: Two main classes



Vision processing with high speed, low power

DynapCNN

Scalable CNN cores

Speck

Integrated vision sensing



HDK



HDK

Smart visual wake-up
Object tracking
Presence detection

Real-time motion estimation
Behaviour detection
Gesture interaction

Natural signal processing

Xylo

Ultra-low-power

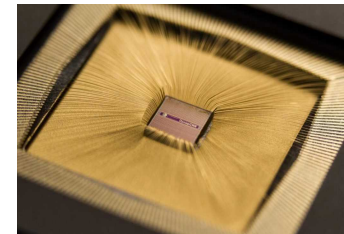


HDK

Audio processing
Bio-signal processing
IMU processing
Condition monitoring

DYNAP™ CNN

- Fully digital, event-driven CNN chip
- Circuit Asynchronous digital circuit
- Designed for spiking CNNs (SCNNs)
- 1M spiking LIF neurons / chip
- Ultra-low power + low latency



SPECK™

- First Fully asynchronous dynamic vision smart sensor
- Single SoC integrates event camera and neuromorphic processor
- Ultra-low power edge computation

Specifications

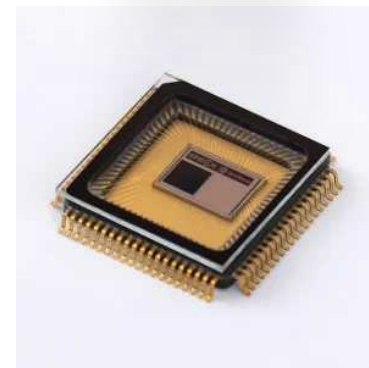
- **Circuit:** Asynchronous digital circuit
- **Neurons:** 320,000
- **Integration:** 19,800/mm²

Resolution: 128*128

Power consumption <5mW

Dynamic range: 90dB

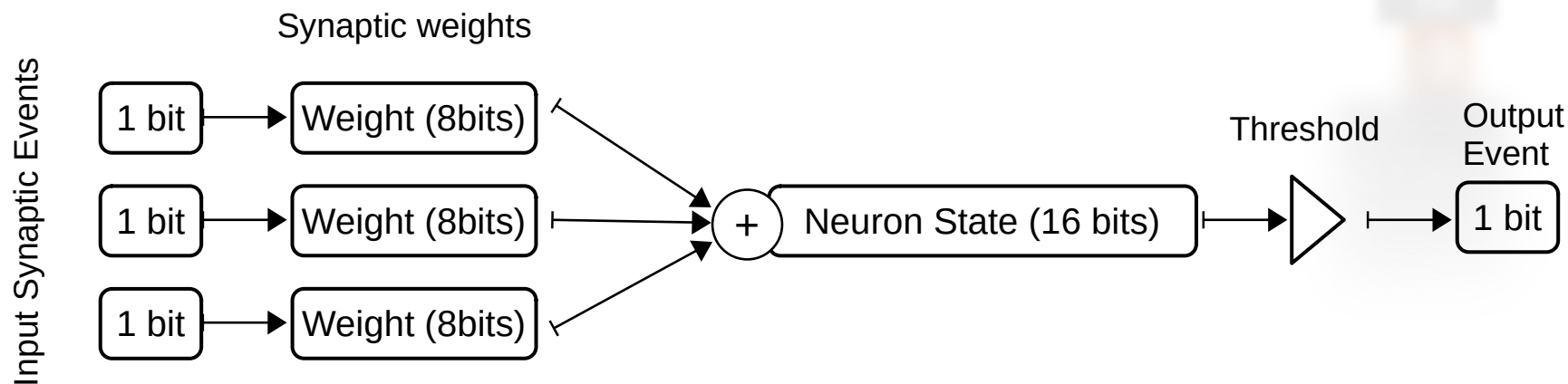
Response time <50ms (typical applications)



SPECK™

DYNAP™ CNN: Neural model

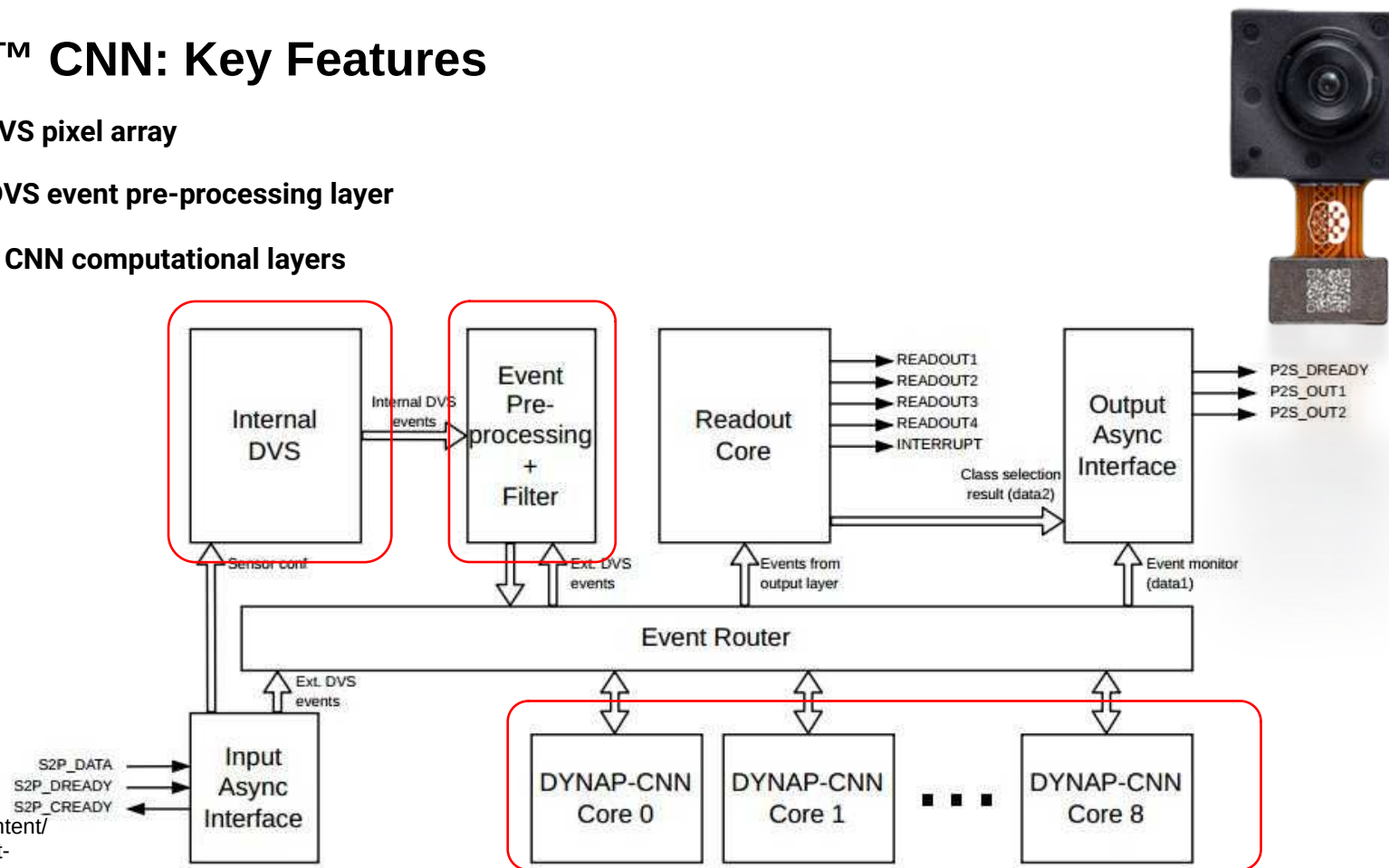
- Integrate and Fire (IAF) model:
 - To lower the complexity
 - Update neuron state only when it receives an event (asynchronously)
 - Less computation and memory update → Lower energy consumption



SPECK™

DYNAP™ CNN: Key Features

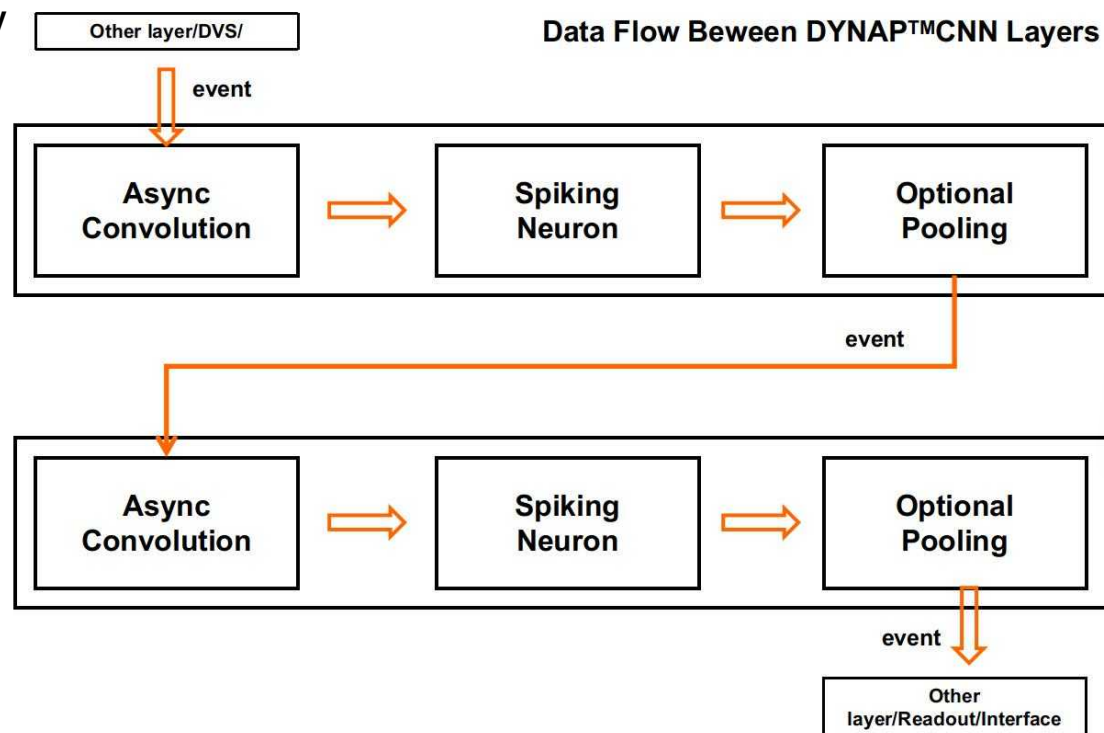
- 128x128 DVS pixel array
- 1 Built-in DVS event pre-processing layer
- 9 DYNAP™ CNN computational layers



SPECK™

DYNAP™ CNN: Layers and Data Flow Pipeline

- Each core (layer):
 - Convolution Connectivity
 - Spiking Functionality
 - Pooling operation



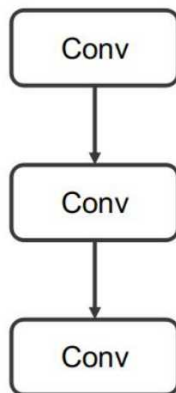
SPECK™

DYNAP™ CNN: Available Network Structure

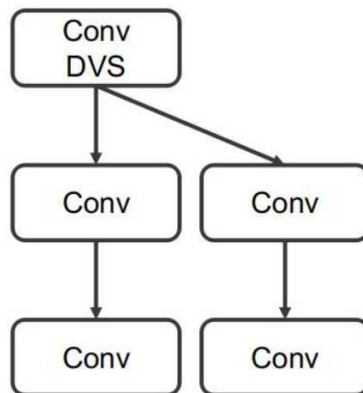
- Each core (layer) supports up to maximally 2 output destinations.

```
# For instance setting output destination from core 0 to core 1 and core 2  
samna_config.cnn_layers[0].destinations[0] = 1  
samna_config.cnn_layers[0].destinations[1] = 2
```

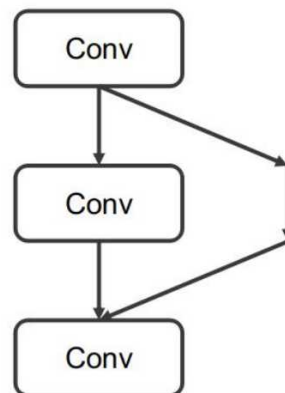
Sequential



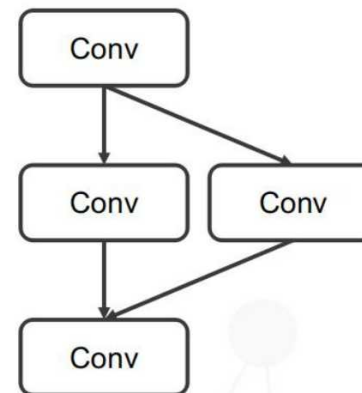
Branched



Residual Connections



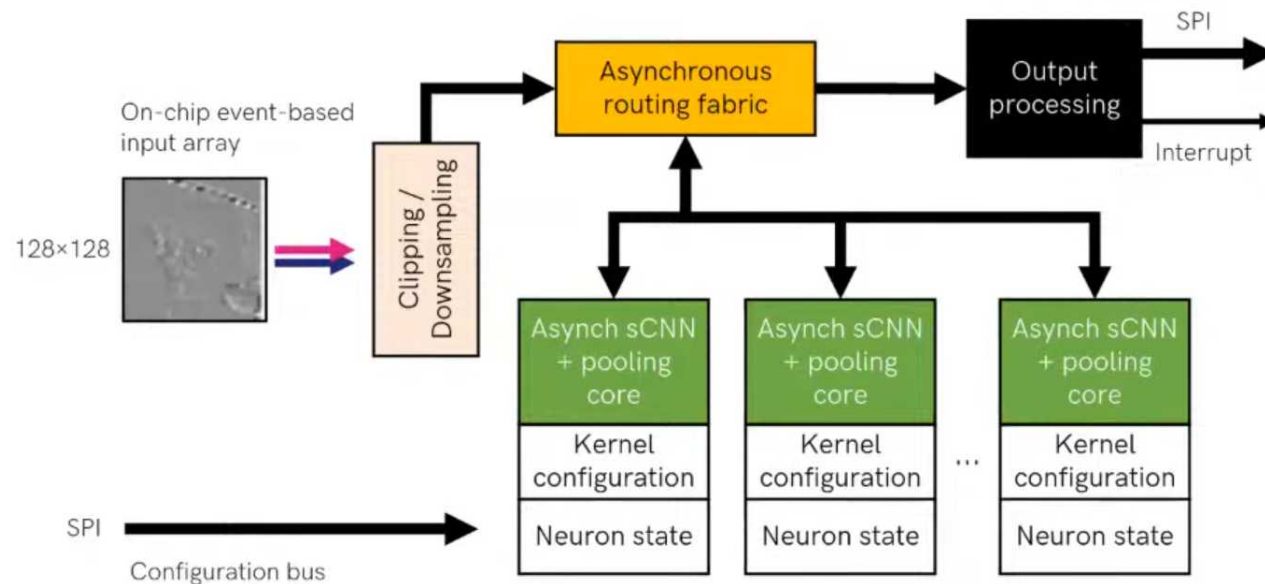
Skip Connections



SPECK™

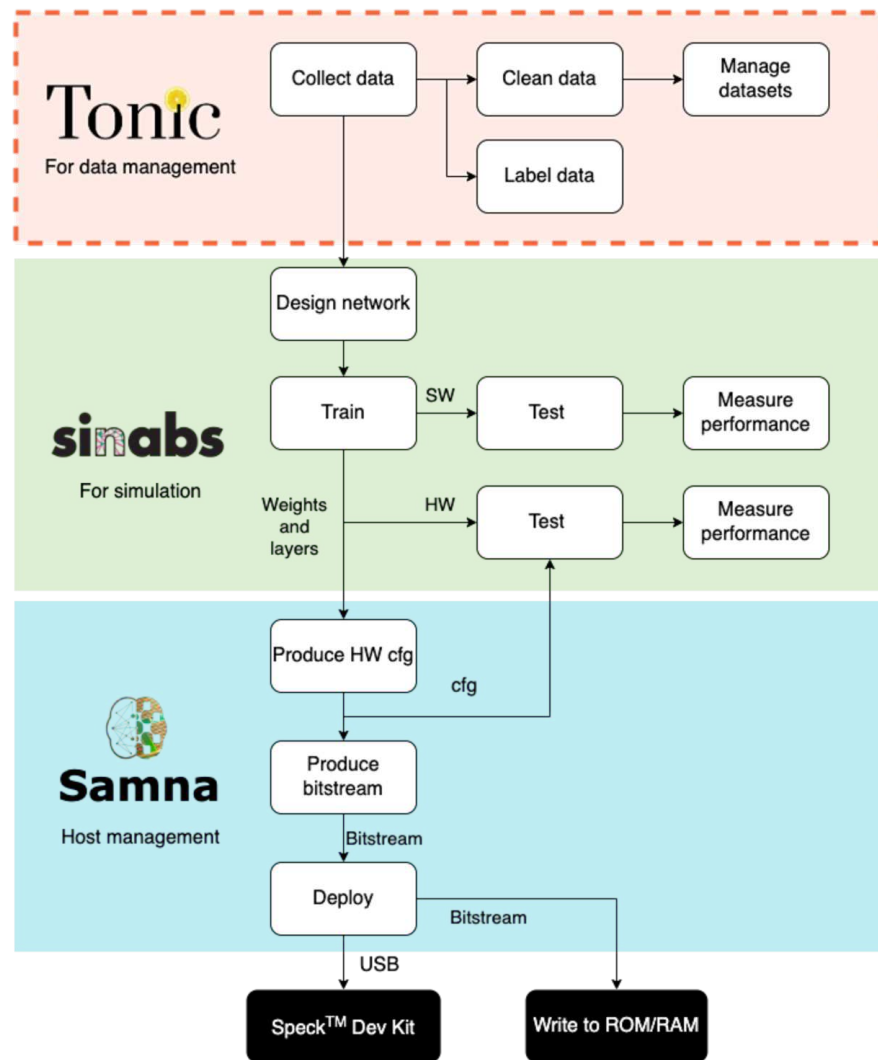
DYNAP™ CNN: Layers and Data Flow Pipeline

- The data flow between core and core are purely based on Address Event Representation(AER) protocol,



SPECK™: Software Tool Chain

- **Tonic:**
PyTorch-ready toolkit for loading, transforming, and using neuromorphic datasets.
- **Sinabs:**
A deep learning framework for designing, converting, and deploying spiking CNNs to neuromorphic hardware.
- **Samna:**
real-time graph-based runtime for configuring, streaming, and controlling SynSense neuromorphic devices.



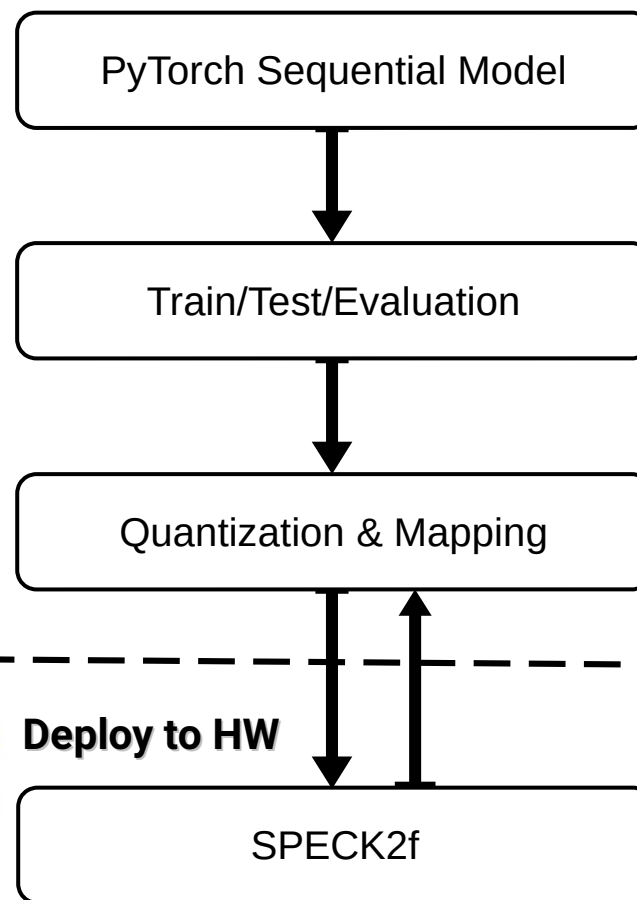
SPECK™: Design & Train Network

sinabs

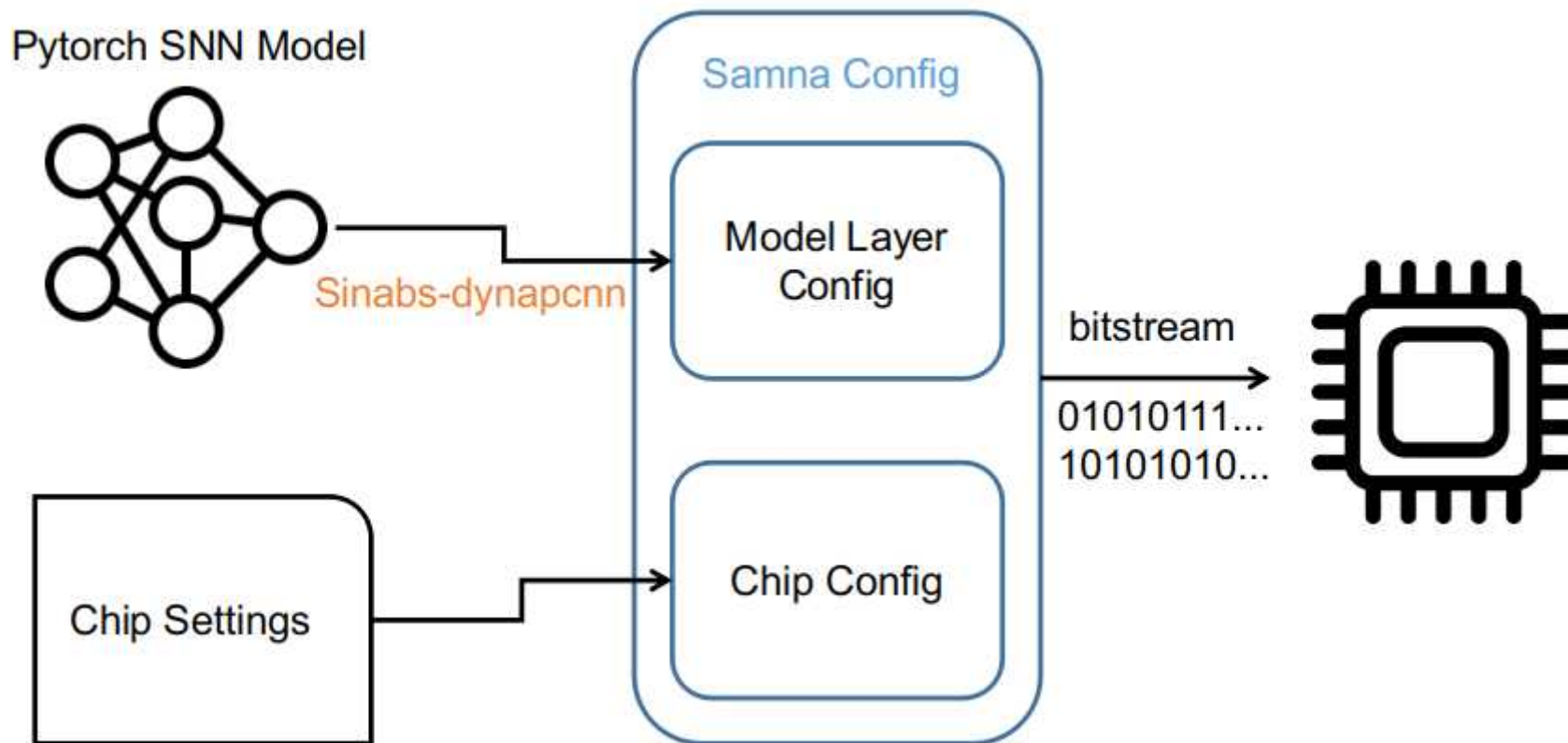
sinabs.dynapcnn



Deploy to HW



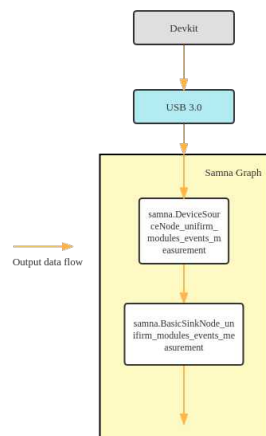
SPECK™: Design & Train Network



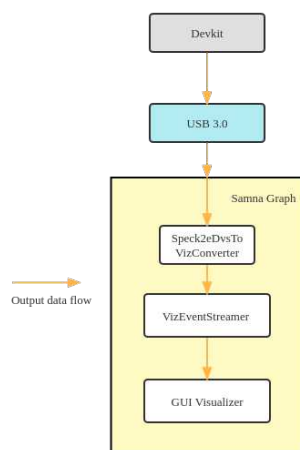
SPECK™: samna graph and filters

samna graph defines how input and output event streams flows between the devkit and the host machine.

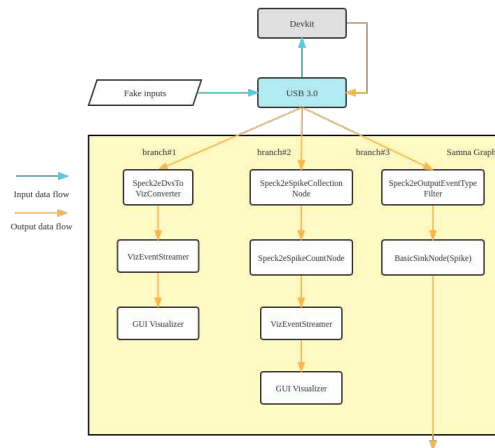
power monitoring



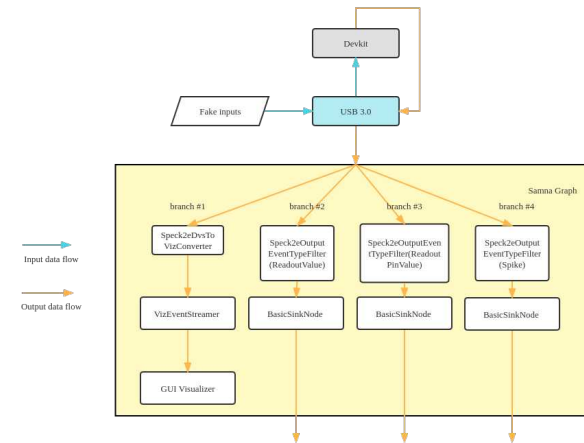
Visualize DVS Input



Spike Count Visualization



Using Readout Layer



SPECK™

You can find more at:

- <https://symsense-sys-int.gitlab.io/samna/0.48.3/index.html>
- <https://sinabs.readthedocs.io/v3.1.3/>
- <https://www.symsense.ai/wp-content/uploads/2025/12/Speck-Dev-Kit-Manual-2025.12-V2.pdf>
- <https://youtu.be/kOiyRtvP02Q?si=YR3JeZYYiERmSzn7>
- <https://github.com/symsense>